Experiment No. 12

Identify XSS Vulnerabilities in Web Applications

AIM

To identify Cross-Site Scripting (XSS) vulnerabilities in a target web application by injecting test payloads into input fields and observing whether they execute in the browser, using manual testing and tools such as OWASP ZAP.

PROCEDURE

1. Identify all input points in the target application (search boxes, comment forms, URL parameters) that reflect or store user input.
2. Manually test each input field with a basic XSS payload, e.g., alert('XSS'), and submit the form/request.
3. Observe the application's response: if the payload executes as a JavaScript alert box rather than being displayed as plain text, the field is vulnerable to reflected or stored XSS.
4. Test URL parameters directly by appending payloads, e.g., search.php?query=alert(1), and check the rendered page source.
5. Use OWASP ZAP's Active Scan (Experiment 9) with XSS-specific rules enabled to automatically fuzz all parameters with multiple XSS payload variants.
6. For stored XSS, submit the payload through a persistent field (e.g., a comment box) and check whether it executes every time the page is reloaded or viewed by another user.
7. Document every vulnerable parameter, the payload used, and whether the XSS is reflected, stored, or DOM-based.

SAMPLE OUTPUT

```
Manual test on search.php:
Request : GET /search.php?query=<script>alert('XSS')</script>
Response: Page displays a JavaScript alert box showing 'XSS'
Finding : Reflected XSS confirmed on parameter 'query'

Manual test on comments.php (stored XSS):
Payload submitted: <img src=x onerror=alert('Stored-XSS')>
Observation: Alert box triggers every time the comments page is reloaded
Finding: Stored XSS confirmed in the 'comment' field

OWASP ZAP Alert: [High] Cross Site Scripting (Reflected) - /search.php,
                 [High] Cross Site Scripting (Persistent) - /comments.php
```

RESULT

Both reflected XSS (in the search parameter) and stored XSS (in the comments field) vulnerabilities were successfully identified in the target web application through manual payload injection and automated scanning with OWASP ZAP.